NATS CLI Cheatsheet

Overview

Download: *https://github.com/nats-io/natscli*

One stop command line utility to interact with and manage NATS.

Independent from NATS server version. Uses only public Golang API.

Please update regularly or compile the GitHub main branch.

Getting started

Built-in cheat sheet

```
nats cheat
```

Always check help for additional options

```
nats server info --help
```

Quick system status

```
nats server list
```

Show raw nats cli messages. No secret sauce

```
nats server list --trace
```

Publish/Subscribe

Listen to a subject

```
nats sub foo
```

Publish to a subject

```
nats pub foo Hello
```

Listen to wildcards

```
nats sub '>'
nats sub 'orders.emea.*'
nats sub 'orders.*.o1234'
```

Context and credentials

List local connectioncontexts

```
nats context list
```

Store credentials and server URL for *cloud*

```
nats context add --server
nats://mycloud.io --user
admin --password admin cloud
```

Select context

```
nats context select cloud
```

Request/Reply

Simple reply mockup

```
nats reply bar World
```

Send a request

```
nats request bar Hello
```

JetStream

List streams, including offline and inaccessible streams

```
nats stream list
```

Create stream interactively and with defaults

```
nats stream add <stream>
nats stream add <stream>
--replicas 3 --defaults
```

Stream view content

```
nats stream view <stream>
```

Publish JetStream message with ack

```
nats pub --jetstream foo Hello
```

Key-Value Store

List Key-Value buckets

```
nats kv list
```

Create a KV bucket

```
nats kv add <bucket>
```

Put and get a key

```
nats kv put <bucket> foo Hello
nats kv get <bucket> foo
```

Watch for changes

```
nats kv watch <bucket> '>'
```

Distributed security

Create an operator

```
nats auth operator add
my-system
```

Generate a sample config

```
nats server generate
server.conf
```

Walkthrough with accounts and users

```
nats auth cheat
```

Server Health and Monitoring

Show system health (SYS)

```
nats server list
```

Detailed server information (SYS)

```
nats server info <server>
```

Check account traffic

```
nats traffic
```

Gather and analyze for debugging (SYS)

```
nats audit gather
nats audit analyze <audit zip>
```

JetStream Monitoring

Show JetStream and meta status (SYS)

```
nats server report jetstream
--watch=1
```

List all streams

```
nats stream report
```

Stream details

```
nats stream info <stream>
```

JetStream backup and purge

Backup a stream with consumer state

```
nats stream backup <stream> <dir>
```

Restore stream from backup

```
nats stream restore <file>
```

Backup just messages

```
nats stream backup --no-consumers
<stream> <dir>
```

Backup all streams in current account

```
nats account backup  <dir>
```

Purge a stream

```
nats stream purge <stream>
```

Purge a subject in a stream

```
nats stream purge --subject foo
<stream>
```

Trace and debug message flow

Trace message without delivering to clients

```
nats trace foo Hello
```

Trace with delivery and timestamps

```
nats trace --timestamp --deliver foo
```

Measure roundtrip and latency

Roundtrip single client to server

```
nats --server srv1:4222 rtt
```

Measure latency between clients connected to different servers

```
nats latency --server srv1:4222
--server-b srv2:4222
```

Benchmarking

Subscribe core NATS for 5 clients

```
nats bench sub foo --clients 5
--msgs 10000
```

Publish core NATS with 10 publishers

```
nats bench pub foo --clients 10
--msgs 10000 --size 512
```

Request-reply server for core nats

```
nats bench service serve --clients 5
testservice
```

Benchmark core nats subscribe for 5 clients

```
nats bench service request --clients
5 testservice --msgs 10000
```

JetStream async acknowledged publishing of batches of 100

```
nats bench js pub foo --create
--batch 100
```

JetStream sync publishing using 10 clients - purge stream

```
nats bench js pub foo --purge
--batch=1 --clients=10 --msgs 10000
```

JetStream delivery from a stream through a durable consumer with 4 instances

```
nats bench js consume --clients 4
--msgs 10000
```

Dashboards - top style

Watch all servers (SYS)

```
nats server watch servers
```

Watch a single server with statistics (SYS)

```
nats top <server-name>
```

NATS CLI Extended

Login options

Login with username and password

```
nats --user <user> --password <pwd> sub foo
```

Login with token

```
nats --token <token> sub foo
```

Login with nkey file - must not contain line breaks

```
nats --nkey <file> sub foo
```

Login with creds file - operator mode - generated with nsc or Synadia Control Plane

```
nats --creds <file> sub foo
```

Storing options in local context

Save connection options to context

```
nats context add LOCAL --server nats://127.0.0.1:4222 --user <user> --password <pwd>
```

Select context

```
nats context select LOCAL
```

Quick validate all contexts (connect servers)

```
nats context validate
```

LLM support

Set `LLMFORMAT=1` in environment to switch to LLM friendly markdown output.

```
export LLMFORMAT=1
```

API Json Schemas

List all supported schemas used by client APIs to communicate with the server. This may include APIs not (yet) exposed in the standard clients.

```
nats schema search
```

Show the schema for the new ***multi get*** API

```
nats schema info io.nats.jetstream.api.v1.stream_msg_get_request
```

List and lookup server errors

List all API error codes emitted by the server

```
nats errors ls
```

Lookup details. The returned `Go Index Constant` is useful for tracing the error in the server code. LLMs in reasoning mode do a good job at identifying code paths leading to the error.

```
nats errors lookup <code>
```

Server Configuration

Obtain current server configuration and statistics (SYS)

```
nats server request variables
```

Reload server config file (SYS) - server id can be obtained with the command above

```
nats server config reload <server id>
```

Server and Stream Events

Connect/Disconnect and authentication errors

```
nats events
```

Including all Jetstream advisories and events

```
nats events --all
```

Graphs - ASCII Dashboard

A number of commands have a `graph` subcommand or `--graph` option.

```
nats server graph
nats stream graph <stream>
nats consumer graph <stream> <consumer>
```

Graph subject traffic. E.g. Jetstream consumer management.

```
nats sub '$JS.API.CONSUMER.CREATE.>' '$JS.API.CONSUMER.INFO.>' '$JS.API.CONSUMER.DELETE.>' --graph
```

Extended Health Check

Health probe (SYS) - used by Kubernetes et. al.

```
nats server report health
```

- Ensure you see all servers and all report 200
- Repeat multiple times if needed

Check clusters and routes (SYS)

```
nats server ls <server count>
```

- Ensure all servers respond quickly. This means all expected servers replied.
- Ensure there are 0 slow consumers
- Ensure routes/GW are consistent
- Look for consistent versions
- Look for consistent CPU usage
- Look for consistent uptime
- Look for consistent memory usage. Is one server using RAM excessivly?
- Check the number of connections.

Message traffic (SYS) - Run as system user for extended info

```
nats traffic
```

- **Raft:** Ensure votes are all to 0 (non zero may indicate instability)
- **Raft:** Ensure append is half of reply
- **General:** Ensure there are less than 1000 JS API Requests per second

Jetstream health - Note that not all servers may have Jetstream enabled (SYS)

```
nats server report jetstream <JS server count>
```

- Ensure all servers respond quickly. This means all expected servers are there.
- Ensure the number of consumers is as expected
- Make sure there is a stable meta leader
- Compare the number of servers in the RAFT Meta Group with Jetstream summary. Ensure all server are current and show in the Jetstream summary.

Gather and analyze information for auditing

```
nats audit gather
```

- Keep audit data for post mortem.

And do a quick analysis

```
nats audit analyze
```

- This is not an exhaustive report. Details may change over time. See `nats audit checks` for a list of current checks.

Exhaustive Stream and consumer status (SYS) WARNING - These reports can be large and impact server performance

```
nats server stream-check
nats server consumer-check
```

- Check for out of sync streams and consumers. Use `--unsynced` to filter.
- Check for expected replicas count, stream size, message count and offline peers

The output of `nats audit gather` can be fed into `stream-check` and `consumer-check` for offline analysis

```
cat ./clusters/*/*/jetstream_info/* | jq -c '.' > jsz.json
nats server stream-check --stdin --unsynced < jsz.json
```

Jetstream queueing/bottlenecks (SYS)

Detecting bottlenecks in Jetstream replications and request routing

```
nats server request ipqueue
```

- Check **Routed JS API Requests** for pending API requests.
- Check RAFT groups for pending operations. Reports account, streams and consumer.

Detailed RAFT status report (SYS)

```
nats server request raftz --account='$G'
```

Identify suspect streams/consumers from `Cluster Group` in `nats stream info`

```
nats server request raftz --account=<account> --group=<raft group>
```

NATS CLI Extended

Configuring accounts and users in operator mode

The nats cli can now replace the `nsc`for most use cases.

- Cleaner and more intuitive syntax.
- More interactive modes for a quick start.
- Uses the same underlying library - *https://github.com/nats-io/jwt*
- Uses the same storage format on disk.

For details see:

- *https://docs.nats.io/running-a-nats-service/configuration/securing_nats/auth_intro/jwt*
- *https://docs.nats.io/running-a-nats-service/configuration/sys_accounts/sys_accounts*

Basic steps for setting up decentralized authentication. All configuration changes are stored locally until **`nats auth account push`**ed to a nats cluster.

Create a new operator with system account

```
nats auth operator add SYSOP
```

Create a new operator with system account

```
nats auth operator add SYSOP
```

Set as default

```
nats auth operator select SYSOP
```

Generate a template server configuration file from an operator

```
nats server generate server.conf
```

Create an application account

```
nats auth account add MyAccount
--defaults
```

Create a new user

```
nats auth account add MyAccount
--defaults
```

Create an admin user in system account

```
nats auth user add admin SYSTEM
--defaults
```

Export credentials for the system user

```
nats auth user credential
sys_admin.creds admin SYSTEM
```

Use `nats context` to set defaults

```
nats context add sysadmin
--description=SYSOP
--server=nats://localhost:4222
--creds=sys_admin.creds
```

Push an account or its changes from a specific operator to a specific server, using system account credentials

```
nats auth account push MyAccount
```

Monitoring with the CLI

nats server check provide simple warning rules, suitable to log file based monitoring. For proper monitoring the **NATS Prometheus exporter** or the **NATS Surveyor** should be used in conjunction with Graphana or similar tools.

List server check options and available warning rules. Checks one server at a time.

```
nats server check
nats server check server --help
```

Simple alerts on server CPU and memory usage (SYS)

```
nats server check server --name
<server> --cpu-warn=50
--cpu-critical=80
--mem-warn=100000000
--mem-critical=200000000
```

Alert on connection count being out of range. The check can be inverted by setting **critical** < **warn** (SYS)

```
nats server check server --name
<server> --conn-warn=1000
--conn-critical=2000
nats server check server --name
<server> --conn-warn=400
--conn-critical=300
```

Report in prometheus format (or nagios, json, text)

```
nats server check server --name
<server> --cpu-warn=50
--cpu-critical=80
--format=prometheus
```

Watch servers (SYS) and jetstream

```
nats server watch servers
nats server watch jetstream
```

Account management

Current account information, connections overview and statistics

```
nats account info
nats account report connections
nats account report statistics
```

Backup and restore all streams in an account.

```
nats account backup <directory>
nats account restore <directory>
```

Purge/Delete all streams in an account. (SYS)

```
nats server account purge
<account>
```

Advanced Jetstream management

Change the cluster leader of a stream or consumer. Useful when stream or consumer are unresponsive

```
nats stream cluster step-down
<stream>
nats consumer cluster step-down
<stream> <consumer>
```

Change to a preferred leader

```
nats stream cluster step-down
<stream> --preferred <server>
```

Rebalance **all** stream leaders in the account - **Use with caution**

```
nats stream cluster balance
```

Scaling a stream down and up

```
nats stream edit <stream>
replicas=3
```

Catching a stream create config json with --json or --trace

```
nats stream add <stream>
--defaults --subject 'foo.>'
--json
nats stream add <stream>
--defaults --subject 'foo.>'
--trace
```

Creating a stream from json config

```
nats stream add --config <config
file>
```

Advanced cluster management

Rebalance client connections between servers (SYS) - **Use with caution** and check the extended options with --help

```
nats server cluster balance
```

Change meta raft leader (SYS) for repairing a cluster

```
nats server cluster step-down
```

Removing a node from a cluster after a cluster scale down (SYS). NATS cluster will wait for nodes to come back. Streams will not repair their replicas until nodes are removed - **Use with caution**

```
nats server cluster peer-remove
<server>
```

If the server name is unknown find and use the server id (SYS)

```
nats server report jetstream
nats server cluster peer-remove
<server id>
```

Created by Synadia, the team behind NATS - www.synadia.com
https://github.com/roeschter/cheat-sheet-nats-synadia - 2026.06.26

NATS CLI Extended

Quick Server status reports

Note: some reports are accessible from the SYS account only or show diffrent details in SYS and app account

The server helth probe as exposed by the HTTP interface and used by K8S et.al. to report server readiness.

```
nats server report health
```

Jetstream status, including meta raft group status.

```
nats server report jetstream
```

Quick memory and cpu usage report

```
nats server report cpu
nats server report mem
```

Detailed server status, profiling and connection kick

Most information is also availabe on the **http debug (port 8222)** interface. Most of this information is auto collected by `nats audit gather`. Except for **connections** all commands require a system account.

List all connections.

```
nats server request connections
```

List connections and all subscriptions

```
nats server request connections
--subscriptions
```

Kick a connection. (SYS)

```
nats server request kick <conn
id> <server id>
```

Current server configuration. (SYS)

```
nats server request variables
```

Detailed status report per server of all streams and replicas. (SYS)

```
nats server request jetstream
```

All subscriptions system wide (SYS)

```
nats server request
subscriptions --detail
```

.. other reports are **acounts**, **gateways**, **leafnodes**, **routes** and **raft**

Various GoLang runtime profiles. (SYS) - Supports: allocs, heap, goroutine, mutex, threadcreate, block, cpu

```
nats server request profile
<profile>
```

CLI Cheatsheet - New Features

Overview

Since Nats-server v2.12 and 2.14 a number of features have been added.
Some have not (yet) been added to the standard API but are published as ***orbit extensions***.

- *https://github.com/synadia-io/orbit.go*
- *https://github.com/synadia-io/orbit.java*
- *https://github.com/synadia-io/orbit.net*
- *https://github.com/synadia-io/orbit.js*
- *https://github.com/synadia-io/orbit.rs*
- *https://github.com/synadia-io/orbit.c*

New stream capabilities

See usage detailed examples in the following sections.

Per-Message TTL - Allows for auto cleaning caches and auto removal of delete markers in KVs.

```
nats s add S1 --defaults --subjects S1 --allow-msg-ttl
```

Atomic Counters in KV stores. Works across sources

```
nats s add S2 --defaults --subjects S2 --allow-counter
```

Scheduled messages - Send messages with delay or on a repeated schedule

```
nats s add S3 --defaults --subjects S3.* --allow-schedules
--allow-msg-ttl
```

Atomic batch - Commit a message batch to a stream atomically. Batch size is limited

```
nats s add S4 --defaults --subjects S4 --allow-batch
```

Fast batch - Send messages in guranteed sequence without waiting for individual acks.

```
nats s add S5 --replicas=3 --defaults --subjects S5 --allow-fast
```

Warning: Only activate features on streams when you intend to use them.

- Some capabilities cannot be disabled once enabled (e.g. schedules)
- Some capabilities are incompatible with Key Value Store or Source and Mirror (e.g. schedules)

Message TTL

Publish a message with diffrent TTLs - see them disappear

```
nats pub S1 Hi --header=Nats-TTL:5s
nats pub S1 Hi --header=Nats-TTL:10s
nats pub S1 Hi --header=Nats-TTL:15s
watch -n 1 'nats s info S1'
```

Atomic Counters

Count (add) to a value atomically. Works on regular streams as well as KVs.

```
nats request S2 '' --header=Nats-Incr:+1
nats request S2 '' --header=Nats-Incr:+2
nats stream get S2 --last-for=S2'
```

Scheduled Messages

Schedule messsages to be sent later.

- All scheduling happens in the same stream.
- The schedule lives in ***unique*** subject it is sent on. The schedule can be deleted, overwritten etc.
- The message is send to `--schedule-dest=SUBJECT`
- The schedule may copy the last message from`--schedule-source=SUBJECT`

Schedule **once** with delay.

```
nats pub S3.1 Hi --schedule-dest=S3.s1 --schedule-at="$(date -d '+10
seconds' -Iseconds)"
nats stream get S3 --last-for=S3.s1'
```

Scheduled message has a TTL.

```
nats pub S3.2 Hi --schedule-ttl=10s --schedule-dest=S3.s2
--schedule-at="$(date -d '+10 seconds' -Iseconds)"
nats stream get S3 --last-for=S3.s2'
```

Schedule `cron` style

```
nats pub S3.3 Hi --schedule-dest=S3.s3 --schedule-cron='*/5 * * * * *'
nats stream get S3 --last-for=S3.s3'
```

Schedule last message from subject

```
nats pub S3.template Hello'
nats pub S3.4 Hi --schedule-dest=S3.s4 --schedule-cron='*/5 * * * * *
' --schedule-source=S3.template
nats stream get S3 --last-for=S3.s4'
```

Atomic Batch Publish

Publish multiple messages in one atomic, all or nothing, operation. Limited to the same stream.

```
echo -e 'A\nB\nC\n' | nats pub --atomic --send-on=newline S4
```

Fast Batch Publish

Allows safe (no dups, no messages lost) fast publishing in order.

```
nats bench js pub fast S5 --stream S5
```

Stream I/O buffer optimizations

Replicated streams do NOT flush their buffers to disk immediately, but rather with a delay or when `sync_interval` triggers. With todays server and OS reliability in memory replication is sufficient guarantee for data persistence over a few seconds interval.
Non replicated stream will flush to disk by default. If performance wins over data consistency replicas=1 streams can be set to async mode

```
nats s add S6 --replicas=1 --defaults --subjects S6
--persist-mode=async
```

CLI Cheatsheet - New Features

Check stream downgrade compatibility

Check the compatibility of streams against past Jetstream API levels. Depends on the feaures actually activated. The command below will show the rquired API level of all streams.

```
nats server report downgrade 0
```

Consumer pause, resume and reset

Consumers can now be paused (for an amount of time), resumed or reset to their initial state or a sequence number.

```
nats consumer create S1 C1 --defaults --pull
nats consumer pause S1 C1 10s
nats consumer resume S1 C1
```

You cannot reset a consumer to a sequence number earlier then the one it started on. But you can set a future one, skipping messages for a while.

```
nats consumer reset S1 C1
nats consumer reset S1 C1 --sequence=42
```

E.g. you can now pause a stuck consumer, find the sequence number of the message it fails to process and then skip over it.

Source/Mirror with durable consumers

Prevously source/mirror relied on ephemeral consumers to initia message stransport, with replication state stored on the mirror stream only.

Ephemeral consumers do not works as intended on interest and workqueue streams and do not allow control to the operator controlling the stream from which messages originate.

2.14 now supports a new flow_control consumer which can be explcitly configured to be used with source and mirror streams.

Create Origin stream with a durable consumer. The consumer is a push consumer, so we need to set a delivery subject.

```
nats s add S_ORIGIN --defaults --subjects S_ORIGIN
nats consumer create --defaults --ack=flow_control --heartbeat=1s
--deliver=all S_ORIGIN C_MIRROR --target mirror.S_ORIGIN.C_MIRROR
```

Create stream. Mirro config accepts a JSON fragment. The permutation of configuration options have become to complex to support as command line parameters.

```
nats s create S_MIRROR --defaults --mirror='{"name":"S_ORIGIN",
"consumer":{"name":"C_MIRROR",
"deliver_subject":"mirror.S_ORIGIN.C_MIRROR"}}'
```